

PySpark + Python

Scenario-Based Interview Questions (Data Engineer) – (2026)

Who is a data engineer?



1. Handling Late-Arriving Data in a Daily Pipeline

Scenario

Your Spark job processes daily transaction data, but sometimes data arrives 1–2 days late. How do you handle this?

Answer

In real projects, late data is common. I handle it using **partition reprocessing + watermark logic**:

1. Store data partitioned by `event_date`
2. Keep last **N days re-processable** (usually 3–7 days)
3. Reprocess those partitions daily

Example:

```
dates = get_last_n_dates(3)

df = spark.read.parquet("/data/events")

filtered = df.filter(col("event_date").isin(dates))

filtered.write.mode("overwrite").partitionBy("event_date").parquet("/processed/events")
```

For streaming:

```
.withWatermark("event_time", "2 days")
```

Why interviewers like this

It shows:

- Production thinking
- Data correctness
- Cost optimization

2. Skewed Data Causing Job Slowness

Scenario

One Spark job is 10x slower. You find one key has millions of records.



Answer

This is **data skew**.

Steps I take:

1. Confirm using:

```
df.groupBy("user_id").count().orderBy(desc("count")).show()
```

2. Solutions:

Option 1 – Salting

```
from pyspark.sql.functions import concat, lit, rand

df = df.withColumn("salt_key", concat(col("user_id"), lit("_"),
                                      (rand()*10).cast("int")))
```

Option 2 – Broadcast small table

```
from pyspark.sql.functions import broadcast
df.join(broadcast(dim_df), "id")
```

Option 3 – Adaptive Query Execution

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

3. Memory Error While Processing Large Files

Scenario

Your job fails with `OutOfMemoryError`.

Answer

I solve it in layers:

1. Reduce data early

```
df = df.select("id", "amount").filter(col("amount") > 0)
```

2. Increase partitions

```
spark.conf.set("spark.sql.shuffle.partitions", 400)
```

3. Persist properly

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

4. Avoid collect()

Never:

```
df.collect()
```



Use:

```
df.show()
df.limit(1000)
```

4. Slowly Changing Dimension (SCD Type-2)

Scenario

Customer address changes. You must maintain history.

Answer

I implement SCD2 using:

- start_date
- end_date
- is_active

Logic:

```
updates = new_df.alias("new").join(old_df.alias("old"), "cust_id")

expired = updates.filter("new.address != old.address") \
    .withColumn("end_date", current_date()) \
    .withColumn("is_active", lit(False))
```

Insert new record:

```
new_records = new_df.withColumn("start_date", current_date()) \
    .withColumn("end_date", lit(None)) \
    .withColumn("is_active", lit(True))
```

5. Removing Duplicates Based on Latest Record

Scenario

Keep only the latest record per user.

Answer

Using **window function**:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window = Window.partitionBy("user_id").orderBy(col("updated_at").desc())

df_latest = df.withColumn("rn", row_number().over(window)) \
    .filter(col("rn") == 1) \
```




```
.drop("rn")
```

6. Processing 1 Billion Records Daily

Scenario

How do you design for scale?

Answer

I focus on:

Area	Approach
Storage	Parquet + Partitioning
Processing	Incremental loads
Joins	Broadcast small tables
Config	AQE + Proper partitions
Pipeline	Airflow orchestration
Validation	Row count & checksum

7. Python UDF vs PySpark Built-in

Scenario

When do you use Python UDF?

Answer

I avoid Python UDF unless:

- Complex ML logic
- Regex heavy logic
- External libraries

Because:

- Serialization overhead
- Slower than JVM

Prefer:

```
from pyspark.sql.functions import when, regexp_extract
```



If needed:

```
@udf(StringType())
def mask_email(e):
    return e.split("@")[0] + "@***"
```

8. Handling Corrupt Records

Scenario

Some JSON records are malformed.

Answer

```
spark.read.option("mode", "PERMISSIVE") \
.option("columnNameOfCorruptRecord", "_corrupt") \
.json("/data")
```

Then store bad records separately:

```
bad_df = df.filter(col("_corrupt").isNotNull())
```

9. Data Quality Validation

Scenario

How do you ensure correctness?

Answer

I implement:

- Row count checks
- Null checks
- Schema validation
- Duplicate checks

Example:

```
assert df.filter(col("id").isNull()).count() == 0
```

10. Spark Job Failed in Production – What Do You Do?

Answer

My real approach:



1. Check logs in Spark UI
 2. Identify stage failure
 3. Retry specific partition
 4. Apply fix
 5. Backfill data
 6. Add alert + monitoring
-

Python Scenario Question

Scenario

You need to read 1000 files efficiently.

Answer

Use multiprocessing or Spark parallelism:

```
from concurrent.futures import ThreadPoolExecutor
```

Or better:

```
spark.read.parquet("/folder/*")
```



1. What is a Data Warehouse? (Interview Definition)

A **Data Warehouse (DWH)** is a **centralized, read-optimized system** designed to store **historical, integrated data** from multiple sources for **analytics, reporting, and decision-making**.

It is **NOT** used for transactions

It is built for **analysis, trends, and KPIs**

One-line interview answer:

A data warehouse stores cleaned, historical business data optimized for analytical queries and reporting.

2. OLTP vs OLAP (MOST ASKED)

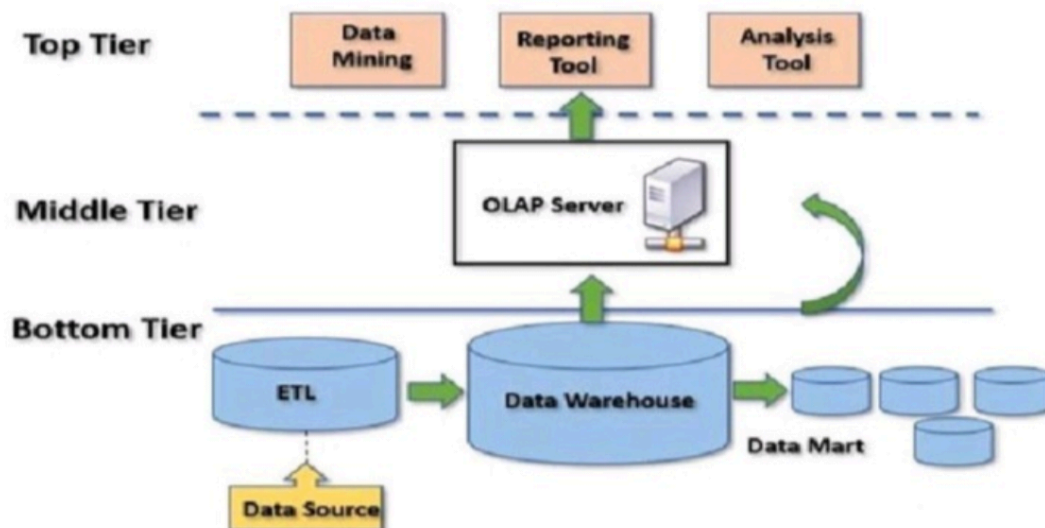
Feature	OLTP	OLAP
Purpose	Transactions	Analytics
Queries	Insert / Update	Complex SELECT
Data	Current	Historical
Tables	Normalized	Denormalized
Example	Order placement	Sales trend analysis

Interview Tip:

If you confuse OLTP & OLAP → interviewer knows fundamentals are weak.

3. Data Warehouse Architecture (3-Tier)





1. Source Layer

- Databases (MySQL, Oracle)
- APIs
- Files (CSV, JSON)
- Streaming sources

2. Staging Layer

- Raw data landing zone
- No transformations
- Used for validation & recovery

3. Presentation Layer

- Fact & Dimension tables
- Used by BI tools (Power BI, Tableau)

4. ETL vs ELT

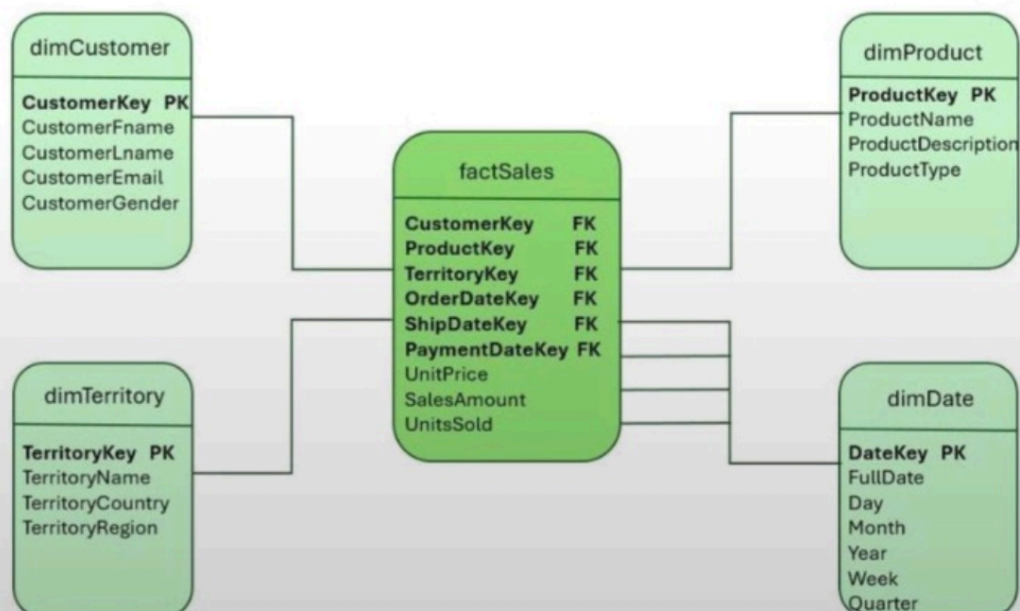
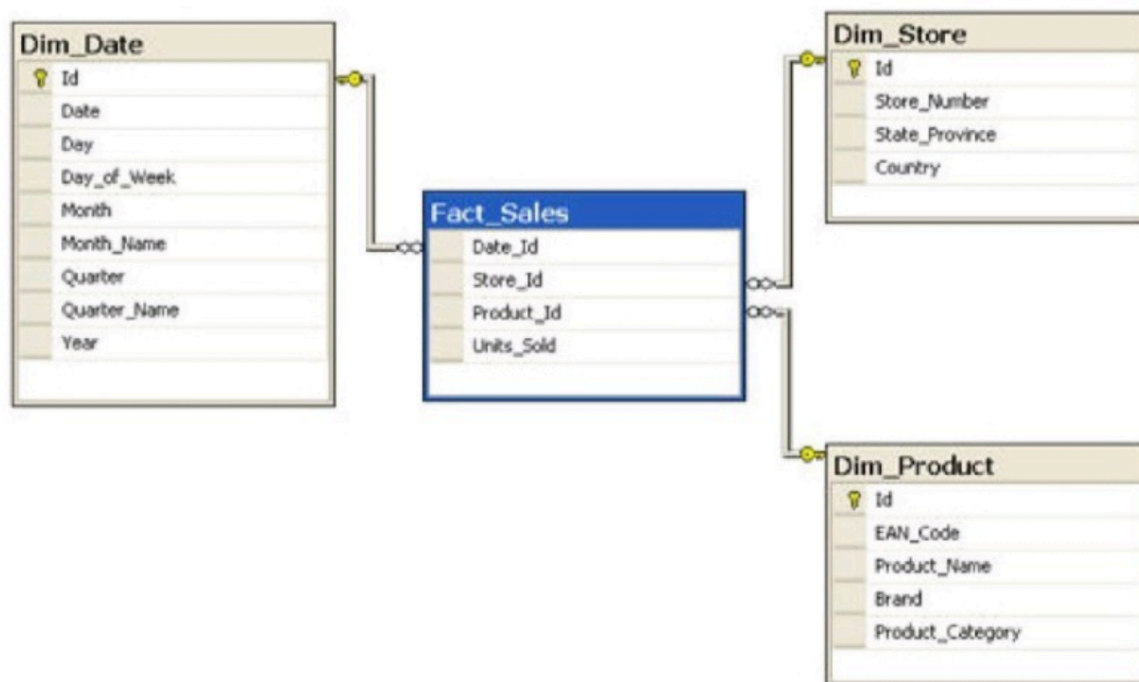
ETL	ELT
Transform before load	Load first, transform later
Used in legacy DWH	Used in cloud DWH
Slower	Faster & scalable



Real Interview Example:

We used **ELT with Snowflake** because transformation was done using SQL after loading raw data.

5. Fact Tables (VERY IMPORTANT)



Types of Fact Tables

1. **Transactional Fact** – One row per transaction



2. **Snapshot Fact** – Daily / monthly snapshots
3. **Accumulating Snapshot** – Process lifecycle tracking

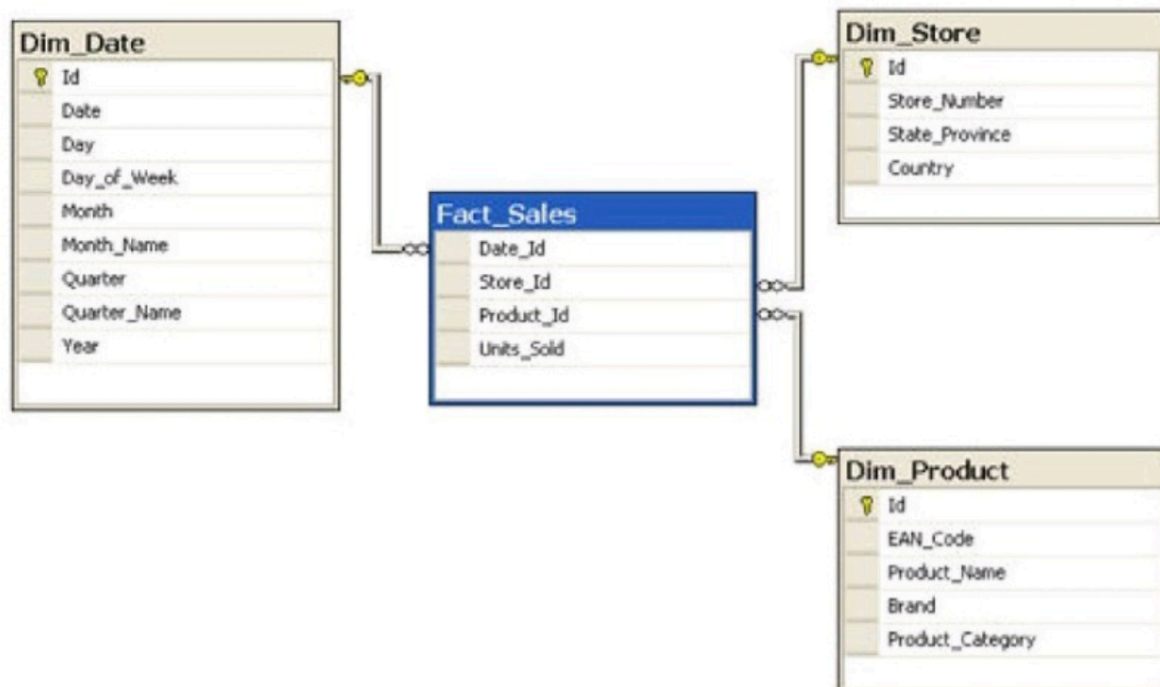
Fact Table Contains:

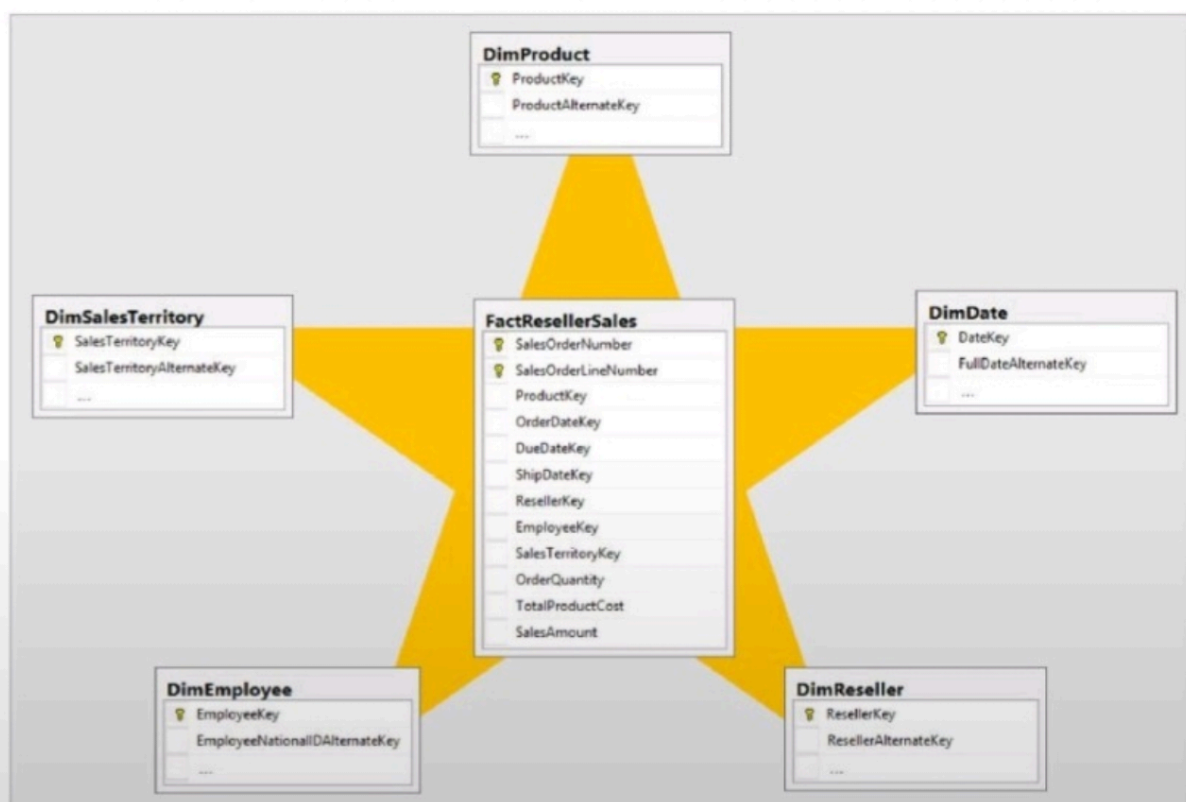
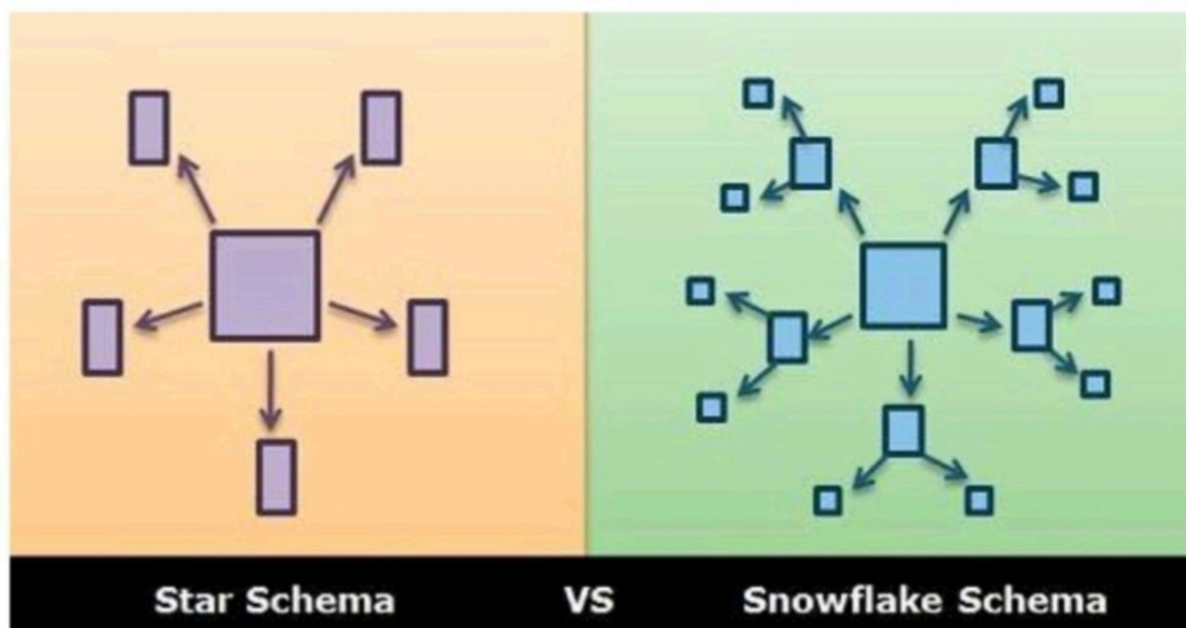
- Measures (Sales, Quantity)
- Foreign keys to dimensions

Interview Trick:

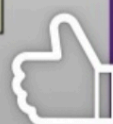
Facts = numeric + measurable

6. Dimension Tables





Star Schema	Snowflake Schema
Simple	Complex
Faster queries	Slower



Denormalized	Normalized
Preferred	Rarely used

Interview Preference: **STAR SCHEMA**

8. Granularity (INTERVIEW)

Granularity = level of detail stored in fact table

Example:

- Order level → High granularity
- Daily sales → Low granularity

Once granularity is decided → **cannot be changed easily**

9. Surrogate Key vs Natural Key

Surrogate Key	Natural Key
System generated	Business key
Integer	String
Stable	Can change

Best Practice: Always use **Surrogate Keys**

10. Data Warehouse Load Types

1. **Full Load** – Load everything
2. **Incremental Load** – Load only changes
3. **CDC (Change Data Capture)** – Track inserts/updates/deletes

Incremental load is preferred in real projects.

11. Partitioning & Indexing

Partitioning

- Improves query performance



Q5. Why use surrogate keys?

Answer:

Because natural keys can change and affect historical data consistency.

REAL INTERVIEW SCENARIO

Question:

Customer address changes — how do you track history?

Answer:

Use **SCD Type 2** with effective date and current flag.

